Email Campaign Annotator — Kiro Specification Solution Document

Author: Vamshi Damera **Version:** 1.0 **Date:** March 03, 2026 **Project:** Email Campaign Annotator — AWS Amplify + Bedrock Automation

---

Revision History

Version	Date	Author	Changes
1.0	March 03, 2026	Vamshi Damera	Initial release — full Kiro spec package

---

Table of Contents

1. Executive Summary
2. What is Kiro?
3. Project Overview
4. Kiro Specification Package Structure
5. Specs: Requirements and Design
6. Agent Hooks
7. Agent Steering
8. Agent Skills
9. How the Kiro Specs Work Together
10. Development Workflow with Kiro
11. Sprint Roadmap
12. Appendix: File Inventory

---

1. Executive Summary

This document describes the complete Kiro specification package created for the Email Campaign Annotator — a fully automated AWS cloud pipeline that replaces a 13-minute manual email proof creation workflow with a sub-5-minute automated process. The Kiro package provides an AI coding agent with everything it needs to understand, extend, and maintain this system correctly and consistently: formal requirements, a technical design, behavioural hooks, guardrail rules, and reusable skill definitions.

The Email Campaign Annotator processes HTML email files uploaded through a React web application, runs them through a 10-step AWS Lambda pipeline powered by two AWS Bedrock Claude models, and delivers a three-page annotated PDF proof to the user's inbox. The Kiro specification package ensures that any AI agent working on this codebase applies the correct patterns, respects the security constraints, and follows the established architecture without needing to rediscover these decisions from the code itself.

---

2. What is Kiro?

Kiro is an AI-powered IDE developed by AWS that uses a structured specification system to guide AI coding agents. Rather than relying on ad-hoc prompts, Kiro uses a `.kiro/` directory in the project repository containing four types of specification files that together define the complete behavioural contract for the AI agent working on the project.

Kiro Component	Location	Purpose
Specs	`.kiro/specs/`	Formal requirements and technical design documents
Hooks	`.kiro/hooks/`	Automated actions triggered by development events
Steering	`.kiro/steering/`	Persistent rules and context injected into every agent session
Skills	`.kiro/skills/`	Reusable, domain-specific knowledge and implementation patterns

Together, these components transform the AI agent from a general-purpose code generator into a specialised expert on this specific project — one that knows the architecture, the constraints, the naming conventions, and the correct implementation patterns for every component.

---

3. Project Overview

The Problem Being Solved

Email marketing professionals at pharmaceutical agencies spend 10–15 minutes per campaign creating annotated PDF proofs for client review. The manual process requires four separate tools: Outlook or Email on Acid for rendering screenshots, Dreamweaver for HTML source inspection, and Adobe InDesign for building the annotated PDF with lettered callouts for each hyperlink.

The Automated Solution

The Email Campaign Annotator replaces this entire workflow with a single web application upload. The user uploads their HTML email file, optionally provides a subject line and recipient address, and receives the completed annotated PDF in their inbox within 5 minutes.

AWS Architecture Summary

The system is built on five AWS managed services, keeping operational overhead minimal for a team of 5–10 users.

Service	Role
AWS Amplify	React SPA hosting with CI/CD from GitHub
Amazon Cognito	User authentication (email + password, no self-registration)
Amazon API Gateway	REST API with Cognito JWT authorizer
AWS Lambda	Serverless processing (Docker container, 3 GB, 5 min)
AWS Bedrock	Claude 3 Haiku (link classification) + Claude 3 Sonnet (quality review)
Amazon S3	Storage for HTML uploads, PDFs, and job history
Amazon SES	Delivery email with quality score and PDF download link

The 10-Step Processing Pipeline

When a user uploads an HTML email, the Lambda processor executes these steps in sequence:

1. Parse the HTML with BeautifulSoup to extract all hyperlinks and metadata
2. Send links to Bedrock (Claude 3 Haiku) for intelligent classification and labelling
3. Send the HTML and metadata to Bedrock (Claude 3 Sonnet) for a 6-category quality review
4. Capture desktop (1200px) and mobile (390px) screenshots using Playwright + Chromium
5. Overlay lettered callout badges (A, B, C…) on the desktop screenshot using Pillow
6. Build a three-page PDF using ReportLab: review report, annotated desktop, annotated mobile
7. Upload the PDF to S3 and generate a 7-day pre-signed URL

8. Send an HTML delivery email via SES with the quality score, top issues, and PDF link
9. Store a JSON job record in S3 for the user's history
10. Return the job result to the web application for immediate display

---

4. Kiro Specification Package Structure

The complete `.kiro/` directory for this project contains 12 specification files organised into four categories.

```
email-annotator/
└── .kiro/
    ├── specs/
    │   ├── requirements.md          ← 32 formal requirements across 5 categories
    │   └── design.md                ← Technical architecture, data models, deployment guide
    ├── hooks/
    │   ├── pre-commit.md            ← 5-step quality gate before every git commit
    │   ├── post-deploy.md           ← 7-step health check after every CDK/Amplify deploy
    │   ├── on-backend-change.md     ← Module-specific rules triggered on file save
    │   └── on-new-feature.md        ← Pre-implementation checklist for new capabilities
    ├── steering/
    │   ├── project-context.md       ← Technology stack, coding standards, file organisation
    │   ├── bedrock-rules.md         ← Model selection, prompt standards, cost monitoring
    │   └── security-rules.md        ← 18 non-negotiable security rules across all layers
    └── skills/
        ├── bedrock-prompt-engineering.md  ← Prompt templates, testing methodology, RAG patterns
        ├── mcp-integration.md             ← 8 MCP integration patterns across 4 sprints
        ├── pdf-generation.md              ← PDF structure, colour standards, fallback patterns
        └── aws-cdk-infrastructure.md      ← CDK patterns for Lambda, API, S3, IAM
```

---

5. Specs: Requirements and Design

5.1 Requirements Specification (`specs/requirements.md`)

The requirements document contains 32 formal requirements organised into five functional categories and three non-functional categories.

Functional requirements cover the complete user journey from authentication through to email delivery. The authentication requirements (REQ-AUTH-01 through REQ-AUTH-04) establish that Cognito is the sole authentication mechanism, self-registration is disabled, and all API endpoints require a valid JWT. The upload requirements (REQ-UPLOAD-01 through REQ-UPLOAD-04) define the file validation rules and the immediate job ID response pattern. The processing requirements cover HTML parsing, Bedrock classification, Bedrock review, screenshot generation, image annotation, and PDF generation in detail. The delivery requirements specify the SES email format and the 7-day pre-signed URL expiry.

Non-functional requirements establish the performance targets (sub-5-minute end-to-end, sub-3-second page load), the security baseline (TLS 1.2, SSE-S3, no public S3 access), the availability target (99.9% leveraging AWS managed services), and the cost ceiling ($5/month for 100 campaigns).

The requirements document also explicitly defines what is out of scope for version 1, including multi-client email rendering, live URL validation, and CRM integration — all of which are addressed in the MCP integration roadmap.

5.2 Technical Design Specification (`specs/design.md`)

The design document translates the requirements into a concrete technical architecture. It defines the component design for all seven system components (React SPA, API Gateway, Processor Lambda, HTML Parser, Bedrock Classifier, Bedrock Reviewer, Screenshot Generator, Image Annotator, PDF Builder, and Jobs Lambda), the data models for job records and API request/response schemas, the complete infrastructure-as-code resource list, the security design including the authentication flow and IAM permissions, and a step-by-step deployment guide.

The design document also includes a forward-looking section mapping each planned MCP integration to the specific module it will update, ensuring that future development follows a clear, pre-designed path.

---

6. Agent Hooks

Hooks are automated actions that the Kiro agent executes in response to specific development events. They enforce quality standards and prevent common mistakes

without requiring the developer to remember to run checks manually.

6.1 Pre-Commit Hook (`hooks/pre-commit.md`)

Trigger: Before every git commit | **Type:** Blocking

This hook runs five sequential checks before any code is committed. Python linting with `ruff` and type checking with `mypy` catch syntax errors and type violations in the backend modules. Secret scanning with `detect-secrets` prevents AWS credentials, API keys, and hardcoded email addresses from entering the repository. The unit test suite runs to ensure no existing tests are broken. CDK synthesis validates that the infrastructure code produces a valid CloudFormation template. Finally, a Vite build check ensures the React frontend compiles without errors.

If any check fails, the commit is aborted and the developer receives specific, actionable error messages. The hook explicitly instructs the agent never to suggest bypassing these checks with `--no-verify`.

6.2 Post-Deploy Hook (`hooks/post-deploy.md`)

Trigger: After every `cdk deploy` or Amplify deployment | **Type:** Advisory

This hook validates the deployed system's health across seven checks. It captures CDK stack outputs, verifies API Gateway CORS preflight, confirms that Cognito self-registration is disabled (a security-critical check treated as blocking), validates S3 bucket access controls, checks Bedrock model availability in the deployed region, verifies SES sender address verification, and displays a reminder to update `aws-config.js` if endpoint values have changed.

The Cognito and Bedrock availability checks are treated as blocking because a deployment where either of these is misconfigured will result in a completely non-functional system.

6.3 On-Backend-Change Hook (`hooks/on-backend-change.md`)

Trigger: When any file in `backend/docker/` is saved | **Type:** Advisory

This hook provides immediate, module-specific feedback when backend pipeline files are edited. For `bedrock_classifier.py`, it verifies the model ID constant, checks

system prompt integrity, and ensures the fallback logic returns the correct list length. For `bedrock_reviewer.py`, it checks the model ID, temperature setting, JSON schema consistency, and HTML truncation limit. For `screenshot_generator.py`, it verifies viewport dimensions and the `networkidle` wait parameter. For `pdf_builder.py`, it checks page order, colour constant consistency, and graceful handling of a missing review. For `handler.py`, it verifies pipeline step order, error response format, CORS headers, and pre-signed URL expiry.

6.4 On-New-Feature Hook (`hooks/on-new-feature.md`)

Trigger: When a new feature branch is created or a new capability is requested | **Type:** Advisory

This hook presents a structured pre-implementation checklist that prevents common mistakes when adding new capabilities. It requires the developer to verify requirements alignment, assess design impact, check whether an MCP server from the roadmap is a better approach than custom code, design Bedrock prompts before writing code (if applicable), write test cases before implementation (TDD), and update all relevant documentation after implementation.

The MCP integration check is particularly important — it prevents the team from building custom code for capabilities that are already planned as MCP integrations in the roadmap.

---

7. Agent Steering

Steering files are persistent rules and context that are injected into every Kiro agent session. Unlike hooks (which are event-triggered), steering rules are always active and define the fundamental operating parameters for the agent.

7.1 Project Context (`steering/project-context.md`)

This is the primary steering file and the first thing the agent reads at the start of every session. It establishes the project's identity and purpose, defines the complete technology stack with rationale for each choice, specifies Python and JavaScript coding standards with concrete examples, articulates the five core architecture principles (serverless first, single responsibility, Bedrock for intelligence, graceful

degradation, least privilege), defines the file organisation structure, and establishes naming conventions for all entity types.

The architecture principles are particularly important for guiding the agent's design decisions. The "Bedrock for intelligence, not rules" principle, for example, prevents the agent from hardcoding link classification logic in Python when that logic belongs in the LLM prompt.

7.2 Bedrock Rules (`steering/bedrock-rules.md`)

This steering file governs all AWS Bedrock usage in the project. It establishes the model selection policy (Haiku for classification, Sonnet for review, no other models without documented justification), defines prompt engineering standards (system prompt requirements, temperature settings, token limits, input truncation), mandates the response validation and fallback pattern for every Bedrock call, provides a cost monitoring table with per-call and monthly estimates, specifies the IAM policy pattern for Bedrock permissions, and previews the two planned Bedrock enhancements (Knowledge Bases MCP for brand-aware review, Data Automation MCP for visual review).

The cost monitoring section is particularly valuable — it gives the agent a concrete cost baseline to reference when evaluating whether a proposed change (e.g., increasing the HTML truncation limit or switching to a more expensive model) is justified.

7.3 Security Rules (`steering/security-rules.md`)

This steering file contains 18 non-negotiable security rules organised across five domains: authentication and authorization, secrets and credentials, S3 data protection, input validation, and IAM least privilege. Each rule is numbered (SEC-1 through SEC-18) and includes a concrete code example showing the correct and incorrect patterns.

The security rules are designed to be applied without being asked. The agent should check every code change against these rules before suggesting it. Rules SEC-1 (no unauthenticated API access), SEC-2 (no self-registration), and SEC-7 (public access blocked) are treated as the highest priority because violations would immediately compromise the system's security posture.

---

8. Agent Skills

Skills are reusable, domain-specific knowledge modules that the agent can apply when working on specific parts of the codebase. Unlike steering (which is always active), skills are consulted when the agent is working on a specific component or task.

8.1 Bedrock Prompt Engineering (`skills/bedrock-prompt-engineering.md`)

This skill is the authoritative reference for writing and modifying Bedrock prompts. It provides four concrete skills: the link classification prompt pattern with a complete, production-ready system prompt template and guidance for extending the label taxonomy; the quality review prompt pattern with the full system prompt, review criteria for all six categories, and the scoring rubric; the structured output reliability pattern covering JSON fence stripping and the validation + fallback pattern; and a prompt testing methodology with five test cases covering clean emails, broken links, missing compliance, placeholder text, and image-heavy emails.

The skill also documents the RAG context injection pattern for the Sprint 1 Bedrock Knowledge Bases MCP integration, ensuring that when that integration is implemented, the prompt structure is consistent with the existing design.

8.2 MCP Integration (`skills/mcp-integration.md`)

This skill provides implementation patterns for all 8 planned MCP server integrations across 4 sprints. For each integration, it specifies which existing module is replaced or extended, the expected impact on the system, a concrete Python code pattern showing how the MCP client is called, and the CDK and IAM changes required.

The skill also defines general MCP integration guidelines covering error isolation (MCP failures must never abort the pipeline), timeout handling (30-second maximum per MCP call), environment variable naming conventions, and the CDK pattern for adding MCP endpoint configuration.

8.3 PDF Generation (`skills/pdf-generation.md`)

This skill defines the standards for the annotated PDF — the primary deliverable of the entire system. It specifies the three-page structure and the rationale for the page order, the exact colour codes for score display and issue severity (with a cross-reference to the matching colours in the SES email), the callout badge design parameters (28px circle, `#e94560` fill, white letter), the current proportional positioning algorithm for callouts and the future exact positioning approach using Playwright MCP bounding boxes, the legend format, and the required fallback behaviours for every possible missing component.

The fallback behaviours are particularly important — they ensure that the PDF is always generated and delivered even when upstream components fail, which is essential for maintaining user trust in the system.

8.4 AWS CDK Infrastructure (`skills/aws-cdk-infrastructure.md`)

This skill provides concrete CDK patterns for the four most common infrastructure tasks: adding a new Lambda function (with separate patterns for ZIP and Docker deployments), adding a new API Gateway route (with the mandatory `auth_opts` pattern), adding a new S3 prefix with an optional lifecycle policy, and adding Bedrock permissions for a new model. It also documents all CDK stack outputs and provides a complete deployment commands reference including user management commands.

---

9. How the Kiro Specs Work Together

The four Kiro components form an integrated system where each component reinforces the others.

The **specs** establish the ground truth — what the system must do (requirements) and how it is built (design). Every other Kiro component references the specs as the authoritative source of truth. When a hook detects a violation, it references the specific requirement number. When a steering rule establishes a pattern, it references the design section that explains why.

The **hooks** enforce the specs at development time. The pre-commit hook prevents code that violates the requirements from entering the repository. The post-deploy

hook verifies that the deployed system matches the design. The on-backend-change hook catches violations of the design's module-specific constraints immediately, before they propagate.

The **steering** provides the agent with the persistent context needed to make correct decisions without consulting the specs every time. The project context steering ensures the agent always uses the correct technology stack. The Bedrock rules steering ensures every AI call follows the established patterns. The security rules steering ensures no security constraint is ever overlooked.

The **skills** provide the agent with deep, actionable knowledge for specific tasks. When the agent needs to modify a Bedrock prompt, it consults the prompt engineering skill. When it needs to add an MCP integration, it consults the MCP skill. When it needs to add a new CDK resource, it consults the infrastructure skill. The skills prevent the agent from reinventing patterns that have already been carefully designed.

---

10. Development Workflow with Kiro

Starting a New Development Session

When a Kiro agent begins a new session on this project, it reads the steering files first to establish context. The `project-context.md` steering file tells the agent what the project is, what technology stack to use, and how to organise code. The `bedrock-rules.md` and `security-rules.md` steering files establish the non-negotiable constraints.

Implementing a New Feature

When a new feature is requested, the `on-new-feature` hook is triggered. The agent works through the pre-implementation checklist: verifying requirements alignment, assessing design impact, checking the MCP roadmap, designing any Bedrock prompts using the prompt engineering skill, writing test cases, and planning documentation updates. Only after completing this checklist does the agent write implementation code.

Modifying Existing Code

When an existing backend module is modified, the `on-backend-change` hook provides immediate, module-specific feedback. The agent checks the modified file against the module-specific rules (correct model ID, correct viewport dimensions, correct page order, etc.) before suggesting the change.

Committing and Deploying

Before committing, the `pre-commit` hook runs all quality checks. Before deploying, the agent runs `cdk diff` to preview changes. After deploying, the `post-deploy` hook validates the live system's health.

---

11. Sprint Roadmap

The Kiro specs are designed to support the project's evolution across four development sprints. Each sprint introduces new MCP integrations that enhance the system's capabilities without requiring architectural changes.

Sprint 1 — Quality Improvements (Recommended Next)

Sprint 1 introduces three MCP integrations that address the most significant quality gaps in the current system. The Linkinator MCP adds live URL reachability checking, catching broken links that the current HTML-only parser cannot detect. The Playwright MCP replaces the custom screenshot code with a managed browser service, reducing Lambda container size by ~40%. The Bedrock Knowledge Bases MCP makes the AI reviewer brand-aware by injecting relevant brand guidelines and regulatory documents into each review call via RAG.

Sprint 2 — Operations and Reliability

Sprint 2 adds operational visibility and removes the API Gateway timeout constraint. The CloudWatch MCP emits custom metrics for each pipeline stage, enabling dashboards and alarms. The SNS/SQS MCP decouples the upload from the processing via an async queue, eliminating the 29-second timeout limitation and allowing the full 5-minute Lambda timeout to be used.

Sprint 3 — Visual Analysis

Sprint 3 adds a visual review pass using the Bedrock Data Automation MCP. After screenshots are captured, they are passed to Bedrock's multimodal capability for analysis of visual quality issues: contrast failures, broken image layouts, and small tap targets that are invisible in the HTML source.

Sprint 4 — CRM Integration

Sprint 4 connects the system to the agency's CRM (HubSpot or Salesforce Marketing Cloud) via MCP, automatically attaching the annotated PDF to the corresponding campaign record and creating a permanent audit trail.

---

12. Appendix: File Inventory

The complete Kiro specification package contains 12 files totalling approximately 15,000 words of structured specification content.

File	Category	Word Count (approx.)	Key Contents
`specs/requirements.md`	Specs	2,500	32 requirements, constraints, out-of-scope
`specs/design.md`	Specs	3,000	Architecture, components, data models, deployment
`hooks/pre-commit.md`	Hooks	600	5-step quality gate
`hooks/post-deploy.md`	Hooks	700	7-step health validation
`hooks/on-backend-change.md`	Hooks	800	Module-specific change rules
`hooks/on-new-feature.md`	Hooks	600	Pre-implementation checklist
`steering/project-context.md`	Steering	1,200	Stack, standards, principles, naming
`steering/bedrock-rules.md`	Steering	1,000	Models, prompts, costs, IAM
`steering/security-rules.md`	Steering	1,100	18 security rules
`skills/bedrock-prompt-engineering.md`	Skills	1,500	Prompt templates, testing, RAG
`skills/mcp-integration.md`	Skills	1,800	8 MCP patterns across 4 sprints
`skills/pdf-generation.md`	Skills	900	PDF structure, colours, fallbacks
`skills/aws-cdk-infrastructure.md`	Skills	900	CDK patterns, deployment commands

This document is the master reference for the Email Campaign Annotator Kiro specification package. All Kiro files are located in the `email-annotator/.kiro/` directory of the project repository.